CIDECODE 2026 ABSTRACT

Team Name: Team Kavacha
Team Members: Shreya Shenoy , Shreya Patil , Aniyath Amrita Pradeep
University Name: PES University (EC Campus)
Batch of 2024-2028

Hackathon Topic Chosen: APK Threat Analysis with C2 Detection
Automated, Evasion-Resistant Android Malware Forensic Triage to Intercept Financial Fraud, dissect spoofed KYC/banking APKs, Trace Code Genealogy, and Cluster Syndicate C2 Infrastructure with Bilingual Vernacular Reporting for Frontline Law Enforcement (Kannada and English)

Problem Statement

Non-technical field officers across Karnataka lack specialized mobile forensic capabilities to rapidly triage malicious, evasion-resistant scam APKs used to intercept financial OTPs. Furthermore, traditional forensic tools evaluate seized apps as isolated incidents, completely failing to link recycled code variants to their overarching, decentralized Command-and-Control (C2) syndicates. This creates massive delays in centralized lab triaging, legal paperwork filing, and state-wide syndicate mapping.

Proposed System Architecture: Project LAKSHYA

Our Project LAKSHYA deploys a **100% Evasion-Resistant Static Forensics Pipeline** that eliminates the need for unstable virtual emulators. It is well known that while testing malware inside an emulator or sandbox it completely freezes.Hence, by analyzing the "dead code" blueprint of a passive APK, the platform extracts high-fidelity threat intelligence and translates it into an actionable, bilingual interface.

Hence dividing the Core of our project into 4 modules each solving a highlight issue is given as follows:

Module 1:Automated Static Reverse-Engineering & Phishing Detection

Instead of dynamically executing volatile malware inside emulators where anti analysis tricks trigger a freeze when detected , LAKSHYA's backend safely decompiles and deploys an automated static reverse engineering pipeline using Androguard to safely decompile compiled dex binaries into readable byte code.

- **Permission Mapping:** Automatically parses the AndroidManifest.xml amd maps structural combinations of it to isolate structural threats , flag high-risk combinations commonly used in financial fraud (e.g., matching RECEIVE_SMS with background internet access).
- **UI Spoofing Detection and Brand Security:** Runs string and regex matching algorithms through decompiled layout resources (res/layout). By checking for unauthorized, hardcoded brand keywords (e.g., *"State Bank of India Login"*, *"YONO Validation"*) against the app's actual cryptographic author signature, the system catches unauthorized brand impersonation red-handed with a definitive mathematical trace.
- To eliminate operational disruptions caused by False Positives (that is misidentifying safe utility apps as malware) while maximizing True positives to detect precision , we integrate deterministic heuristic evaluation engine.
- **Dynamic Confidence Scoring:** Rather than delivering binary alerts the platform yields a weighted probability score based on permission risks , signature mismatches and structural genealogy matches.
- **Contextual Triage:** To avoid False Positives , if an application requests sensitive SMS access but possesses verified institutional signatures and zero unauthorized trademarked layouts then the system classifies this as a safe utility , dynamically dropping the False positive margin to nearly 0%

Module 2: Code DNA tracking & Call-Graph Fingerprinting

To target the broader fraud syndicate rather than just a single file, LAKSHYA treats malware code like structural DNA.

- **Function Call Graphs (FCG):** The platform maps out the internal structural framework of how code functions interact, converting this topology into a unique numerical vector fingerprint.
- **Genealogy Comparison:** Even if a hacker changes the file name, obfuscates variables, or alters the destination server URL, LAKSHYA compares its structural skeleton against a database of past cases, instantly alerting investigators if a "new" app shares high DNA similarity with an older cybercrime filing.

Module 3: Cross-Case C2 Infrastructure Clustering (Forensic Artifact Extraction)

Instead of analyzing seized devices in isolation, this module uncovers the shared server networks driving decentralized scam campaigns.

- **IOC Extraction:** The pipeline strips hardcoded URLs, IP addresses, and custom communication ports out of the decompiled code.
- **Graph-Based Clustering:** Extracted markers are cross-referenced with live threat intelligence feeds (using *URLhaus, ThreatFox, AbuseIPDB*). Using network graph modeling, the system clusters independent cases; if an APK uploaded in Bengaluru and another from Mysuru communicate with the same IP range or registrar, the system visually maps them to a singular threat actor group hence mapping various cases which on the broader picture seemed to have no correlation whatsoever .
- Using NetworkX, the platform maps out an interactive **Network Flow Visualization** to show shared C2 clusters at a glance

Module 4: Last-Mile Vernacular Reporting (The Human Angle) and Forensic Artifact Export

Designed specifically to democratize advanced cybercell forensics, this module removes technical barriers for frontline field officers.

- **Bilingual Triage Dashboard:** Converts complex forensic logs into a simple, high-contrast risk badge accompanied by a clear summary in both **English and Kannada** (e.g., *"ಈ ಅಪ್ಲಿಕೇಶನ್ ನಿಮ್ಮ ಬ್ಯಾಂಕ್ ಒಟಿಪಿ (OTP) ಮತ್ತು ಸಂದೇಶಗಳನ್ನು ಕದ್ದು ಹ್ಯಾಕರ್‌ಗೆ ಕಳುಹಿಸುತ್ತದೆ" / "This app steals your Bank OTP messages and sends them to the hacker"*).
- **FIR-Ready Evidence Output:** The platform auto-generates structured, legally sound technical summaries formatted to be immediately attached to a First

Information Report (FIR) or charge sheet, which count as structure forensic reports thereby completely eliminating paperwork delays at local police stations.

Technical Stack

1)Frontend: ReactJS for a high-contrast, military-grade dashboard displaying interactive network graphs, similarity percentages, and bilingual reporting. The UI translates the extracted permission profiles into a clear behavior timeline visualizing what actions the app attempts sequentially.

2)Backend Forensic Pipeline: Python (FastAPI/Flask) utilizing specialized static analysis engines (Androguard, Apktool) for deep binary-parsing without risk of malware execution.

3)Graph & Data Modeling Layer: NetworkX to calculate structural code vector matching and map interconnected C2 infrastructure dependencies.

4)Bilingual Translation & Summarization Engine: RAG-augmented localized language processing models to securely translate technical security flags into operational, legally sound English and Kannada text templates.

Technical Feasibility (24-48 Hour Build)

Stable Core Engines: Uses mature Python forensics libraries (Androguard, Apktool) to parse raw files safely without execution.
Lightweight Architecture: Eliminates unstable emulators and sandboxes, reducing compute overhead to basic, fast static string and regex matching.
Deterministic Localization: Combines localized dictionary overlays with structured templates for reliable, legally sound Kannada and English FIR copy generation.

Execution Timeline

1–3 hr: Backend API setup and file-ingestion pipeline for raw .apk uploads.

3–6 hr: Static parsing engine implementation (AndroidManifest extraction and C2 URL regex parsing).
6–9 hr: Code genealogy mapping and cross-case network clustering database logic.
9–12 hr: ReactJS dashboard development displaying interactive graphs and bilingual cards.
12–15 hr: Automated PDF compilation engine for evidence-ready FIR briefs.
15–18 hr: Testing pipeline against twin safe/malicious datasets and refining live demo slides.
Final 6 hours(buffer): Preparing for the pitch and PPT

Risk Handling & Mitigation

Code Obfuscation: Hackers rename functions to hide code.
Mitigation: Track structural *Function Call Graph topology* (how functions connect) instead of text names.
Data Privacy: Public cloud exposure of evidence is a security risk.
Mitigation: The static parsing and dictionary lookups operate strictly on a self-contained local backend.
Demo Connectivity Drop: Live threat-intelligence API queries might choke during judging.
Mitigation: System implements a pre-cached local database of known malicious C2 signatures for offline fallback.

Bureau Scope

1)Empowers Local Police Stations by getting advanced forensic capabilities to small, local stations, allowing field constables to handle complex mobile scam cases instantly without waiting for specialized labs increasing the delay
2) Busting Crime Rings by connecting multiple separate local complaints to a single central server infrastructure
3) Remove Manual Paperwork by writing technical evidence in both Kannada and English, cutting down an officer's file-drafting time from several hours
4) Bridges the Language and Tech Barrier :Translates complex malware code into a simple, bilingual (Kannada + English) 1-page dashboard, enabling any non-technical officer to triage cybercrime evidence instantly.

Resources Referred :

1) https://jset.sasapublications.com/wp-content/uploads/2019/03/1961002.pdf
2) https://philpapers.org/rec/KAMDMA
3) https://link.springer.com/chapter/10.1007/978-3-031-09484-2_3
4) https://arxiv.org/abs/2601.08959
5) http://polipapers.upv.es/index.php/IJPME/article/view/1502

_____ END_____